

HW 1 (3 pages)

Horizon 529839

Problem I): (for Lecture 1)

Solve 1--3 by using Python as a calculator, which is easier to run using Spyder (which is installed at the same time you install Jupyter). Then solve 4—5 by using our Python program L1_IBM.ipynb with the necessary modifications (remember using the “Run” command to run the programs after editing in the Jupyter environment).

1. Compute the present value and discount futures cash flows (with simple compounding):

a) Type $100 \times (1+0.15)$ at the console (bottom right), and hit Enter;

```
>>> 100*(1+0.15)
114.99999999999999
```

b) Type $r=.15$, Enter;

```
>>> r = .15
>>> r
0.15
```

c) Type $100 \times \text{pow}(1+r,1)$, Enter; [you get the value $100 \times (1+r)$]

```
>>> 100*pow(1+r,1)
114.99999999999999
```

d) Type $100 \times \text{pow}(1+r,10)$, Enter; [you get the value $100 \times (1+r)^{10}$]

```
>>> 100*pow(1+r,10)
404.5557735707907
```

e) Type $100 \times \text{pow}(1+r,-2)$, Enter; [you get the value $100/(1+r)^2$]

```
>>> 100*pow(1+r,-2)
75.61436672967865
```

f) How do you compute the present value of 10 payments:

$100/(1+r) + 100/(1+r)^2 + \dots + 100/(1+r)^{10}$, enter return.

(Hint: set $\text{sum}=0$; use a loop: `for i in range(10): sum=sum + ?`)

```
sum = 0
for i in range(1,11):
    sum += 100*pow(1+.15,i)
print(f"The result for '100/(1+r) + 100/(1+r)^2+...+ 100/(1+r)^10' is {sum}")
```

```
The result for '100/(1+r) + 100/(1+r)^2+...+ 100/(1+r)^10' is 2334.927597376063
```

```
Process finished with exit code 0
```

2. Do the same problem with continuously compounding. Now you need to import the exponential function by typing: `import math`, Enter. Then `math.exp(.)` is the exponential function.

```
>>> import math
>>> math.exp(1)
2.718281828459045
```

3. Compute a portfolio return with provide the equal weights on 3 stocks whose returns are 10%, -5% and 20%, respectively.

[Hint: You need type in: `import numpy as np` (to use array function). Then type in `w = np.array([1/3,1/3,1/3])`, and `R = np.array([.10,-.05,.20])`, to create the vectors, and finally compute the dot product, `port = np.dot (w,R).`]

(We will program more realistic and complex cases in the future)

```
import numpy as np
w = np.array([1 / 3, 1 / 3, 1 / 3])
R = np.array([.10,-.05,.20])
port = np.dot (w,R)
print(port)
0.08333333333333333
```

Process finished with exit code 0

4. Compute the average return of the IBM stock by using the loops (without using mean command) over the two sub-periods: Jan 1934 to December 2006, and Jan 2007 to December 2011. (Hint: the easiest way is to change the Do loop from t1 to t2.)

```
import pandas as pd
def cal_er(indices,datasource):  2 usages
    sum_return_1 = 0
    for i in indices:
        sum_return_1 += datasource[i]
    average_return_1 = sum_return_1 / len(indices)
    return average_return_1

def q_f():  1 usage
    filepath = "/Users/yuan_oli/Documents/Data Analysis for investment/HW/HW_1/Python1/"
    data_name = "IBM.csv"
    df = pd.read_csv(filepath + data_name)  # note: To write a data to cvs
    Re = df.loc[:, "Return"]
    indices = df[(df["Date"] > 19340131) & (df["Date"] < 20061229)].index.tolist()
    indices_2 = df[(df["Date"] > 20070131) & (df["Date"] < 20111230)].index.tolist()
    ave_r1 = cal_er(indices,Re)
    ave_r2 = cal_er(indices_2,Re)
    print(f"The average return from Jan 1934 to December 2006 to is {ave_r1*100:.2f}% :")
    print(f"The average return from Jan 2007 to December 2011 to is {ave_r2*100:.2f}% :")

q_f()
```

```
The average return from Jan 1934 to December 2006 to is 1.27% :  
The average return from Jan 2007 to December 2011 to is 1.43% :
```

5. Taking the negative returns as zeros, compute the average return of the IBM stock by using the loops and if statements (without using mean command) over the two sub-periods.

```
import pandas as pd  
def cal_er(indices,datasource): 2 usages  
    sum_return_1 = 0  
    for i in indices:  
        if datasource[i] >0:  
            pass  
        else:  
            datasource.iloc[i] = 0  
        sum_return_1 += datasource[i]  
    average_return_1 = sum_return_1 / len(indices)  
    return average_return_1  
  
def q_f(): 1 usage  
    filepath = "/Users/yuan_oli/Documents/Data Analysis for investment/HW/HW_1/Python1/"  
    data_name = "IBM.csv"  
    df = pd.read_csv(filepath + data_name) # note: To write a data to cvs  
    Re = df.loc[:, "Return"]  
    indices = df[(df["Date"] > 19340131) & (df["Date"] < 20061229)].index.tolist()  
    indices_2 = df[(df["Date"] > 20070131) & (df["Date"] < 20111230)].index.tolist()  
    ave_r1 = cal_er(indices,Re)  
    ave_r2 = cal_er(indices_2,Re)  
    print(f"The average return from Jan 1934 to December 2006 to is {ave_r1*100:.2f}% :")  
    print(f"The average return from Jan 2007 to December 2011 to is {ave_r2*100:.2f}% :")  
  
q_f()
```

```
The average return from Jan 1934 to December 2006 to is 3.14% :  
The average return from Jan 2007 to December 2011 to is 2.98% :
```

```
Process finished with exit code 0
```

Problem II: (for Lecture 2)

Bases on the program L2_Analysis.ipynb and the data on IBM, S&P500 and riskfree rate from January 1934 to December 2011, answer the following questions over two sub-periods: Jan 1934 to December 2006, and January 2007 to December 2011

1) What are the average monthly return and risk (sample mean and sample standard deviation) for each of the two assets?

```
filepath = "/Users/yuan_oli/Documents/Data Analysis for investment/HW/HW_1/Python1/"
ibm_name = 'IBM.xlsx'
sp_name = 'SP500.xlsx'
risk_free_name = 'Riskfree.xlsx'
df_ibm = pd.read_excel(filepath+ibm_name) # the data is in xlsx format
df_sp = pd.read_excel(filepath+sp_name)
df_rf = pd.read_excel(filepath+risk_free_name)
# Re_ibm = df_ibm.loc[:, "Return"]
Re_sp = df_sp.loc[:, "Return"]
#load the data for IBM
re_ibm_pf = df_ibm[(df_ibm["Date"] > 19340131) & (df_ibm["Date"] < 20061229)].loc[:, "Return"]
re_ibm_ps = df_ibm[(df_ibm["Date"] > 20070131) & (df_ibm["Date"] < 20111230)].loc[:, "Return"] #class 'pandas.core.series.Series'
#load the data for S&P 500
re_sp_pf = df_sp[(df_sp["Date"] > 19340131) & (df_sp["Date"] < 20061229)].loc[:, "Return"]
re_sp_ps = df_sp[(df_sp["Date"] > 20070131) & (df_sp["Date"] < 20111230)].loc[:, "Return"]

def q_one(data_source, start_date, end_date, p_name): 4 usages
    start_date = datetime.strptime(str(start_date), format: "%Y%m%d")
    # print(start_date)
    end_date = datetime.strptime(str(end_date), format: "%Y%m%d")
    mu = np.mean(data_source)
    sig = np.var(data_source) # The variance, i.e., the square of the standard deviation
    std = np.sqrt(sig) # The standard deviation
    print(f"{p_name} for the time period {start_date.date()} to {end_date.date()}: \n The average monthly return is {mu:.5f} and standard deviation is {std:.5f}")

q_one(re_ibm_pf, start_date: 19340131, end_date: 20061229, p_name: "IBM")
q_one(re_ibm_ps, start_date: 20070131, end_date: 20111230, p_name: "IBM")
print('\n')
q_one(re_sp_pf, start_date: 19340131, end_date: 20061229, p_name: "S&P")
q_one(re_sp_ps, start_date: 20070131, end_date: 20111230, p_name: "S&P")
```

IBM for the time period 1934-01-31 to 2006-12-29:

The average monthly return is 0.01273 and standard deviation is 0.06580

IBM for the time period 2007-01-31 to 2011-12-30:

The average monthly return is 0.01430 and standard deviation is 0.05702

S&P for the time period 1934-01-31 to 2006-12-29:

The average monthly return is 0.00989 and standard deviation is 0.04564

S&P for the time period 2007-01-31 to 2011-12-30:

The average monthly return is 0.00106 and standard deviation is 0.05464

2) What is the reward per unit of risk (excess mean/std, known as *Sharpe Ratio*) ?

```
For S&P from 1934-01-31 to 2006-12-29:
monthly Sharp Ratio is 0.14929 and annulaized Sharpe ratio is 0.517142
For S&P from 2007-01-31 to 2011-12-30:
monthly Sharp Ratio is 0.00680 and annulaized Sharpe ratio is 0.023550

For IBM from 1934-01-31 to 2006-12-29:
monthly Sharp Ratio is 0.14605 and annulaized Sharpe ratio is 0.505938
For IBM from 2007-01-31 to 2011-12-30:
monthly Sharp Ratio is 0.22589 and annulaized Sharpe ratio is 0.782500
```

3) Which of the equity assets has a higher *Sharpe Ratio* ?

For time period : Jan 1934 to December 2006

SMP has higher Sharp Ratio.

For time period : January 2007 to December 2011

IBM has higher Shap Ratio

4) If you invest \$1,000 in IBM and S&P500 in January 2007, respectively, what will be your wealth if you hold it to the end of the period?

```
def q_4(data_set,p_name): 2 usages
    #initial the investment amount.
    ini_amo = 1000
    #Calculate the final
    for i in data_set:
        ini_amo = ini_amo * (1 + i)
    print(f"invest 1,000$ in {p_name} from January 2007, at 2011-12-30 will be {ini_amo:.6f}")

#re_sp_ps: monthly return of S&P500 in Second Period :2007-01-31 to 2011-12-30.
#re_ibm_ps: monthly return of IBM in Second Period :2007-01-31 to 2011-12-30.
q_4(re_sp_ps, p_name: 'S&P500')
q_4(re_ibm_ps, p_name: "IBM")
```

```
invest 1,000$ in S&P500 from January 2007, at 2011-12-30 will be 998.072687
invest 1,000$ in IBM from January 2007, at 2011-12-30 will be 2065.708825
```

5) What are the skewness and kurtosis of IBM and S&P500?

```
def q_5(data_set,start_date,end_date,p_name): 4 usages
    skew = 0 # initialize it be zero
    kurt = 0
    start_date_5 = datetime.strptime(str(start_date), format: "%Y%m%d")
    end_date_5 = datetime.strptime(str(end_date), format: "%Y%m%d")

    T = len(data_set) # Get the length, # of obvs (the headers of the Excel doesn't count)
    mu,sigma = q_one(data_set,start_date,end_date,p_name) #Call the question 1 part to get the sigma and mu for the input dataset

    for i in data_set:
        skew = skew + pow(i - mu, 3) # sums the 3rd power terms successively
        kurt = kurt + pow(i - mu, 4)

    skew = (skew / pow(sigma, 3)) / T # take the average
    kurt = (kurt / pow(sigma, 4)) / T

    print(f'\n{p_name} for the time period {start_date_5.date()} to {end_date_5.date()}:\n (Monthly) from skew is {skew:.6f}, skew is {kurt:.6f}')

#call function to do calculation for each period
q_5(re_ibm_pf, start_date: 19340131, end_date: 20061229, p_name: "IBM")
q_5(re_ibm_ps, start_date: 20070131, end_date: 20111230, p_name: "IBM")
print('')
q_5(re_sp_pf, start_date: 19340131, end_date: 20061229, p_name: "S&P")
q_5(re_sp_ps, start_date: 20070131, end_date: 20111230, p_name: "S&P")
```

IBM for the time period 1934-01-31 to 2006-12-29:
(Monthly) from skew is 0.291084, kurt is 5.078594

IBM for the time period 2007-01-31 to 2011-12-30:
(Monthly) from skew is -1.081815, kurt is 5.549943

S&P for the time period 1934-01-31 to 2006-12-29:
(Monthly) from skew is -0.437278, kurt is 6.670246

S&P for the time period 2007-01-31 to 2011-12-30:
(Monthly) from skew is -0.488582, kurt is 3.357375

6) Is S&P500 closer to the normality assumption ?

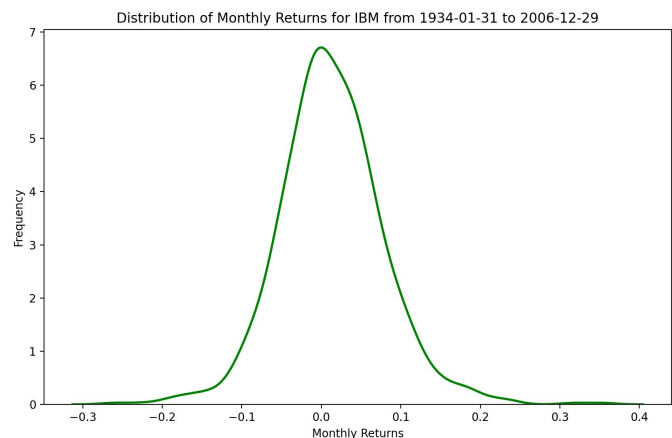
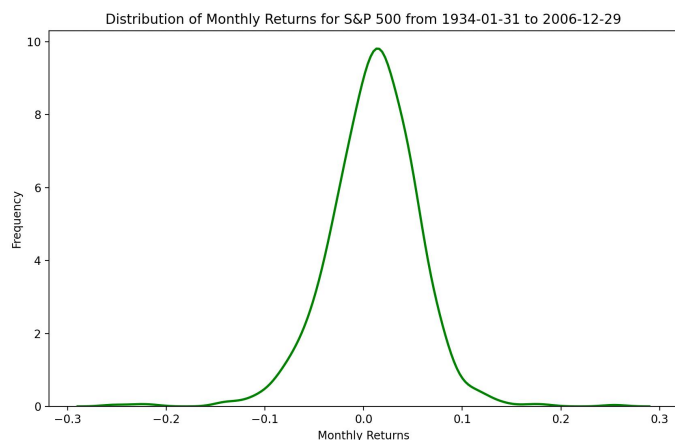
Answer:

According to the data from question 5, the S&P 500 is closer to the normality assumption in the 2007-2011 period compared with IBM, since the S&P 500 skewness is closer to 0 and kurtosis closer to 3. However for the time period 1934 to December 2006, IBM is closer to the normality assumption than the S&P 500.

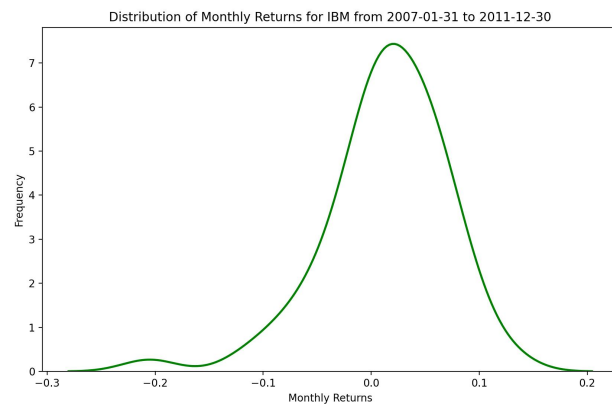
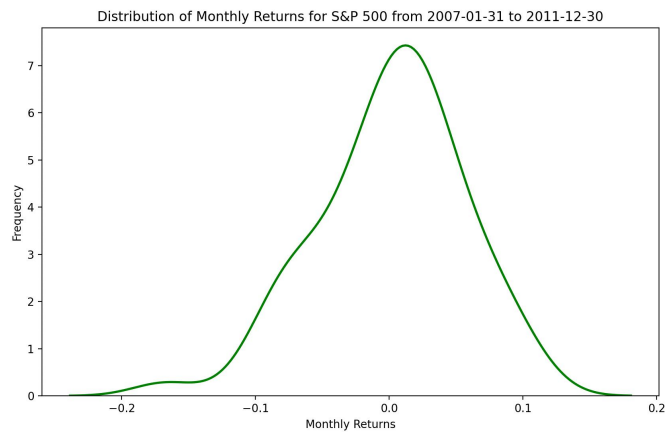
```
def q_6(): 1 usage
    plt.figure(figsize=(10, 6))
    sns.kdeplot(re_ibm_ps, color='green', linewidth=2)
    # sns.histplot(re_sp_pf, bins=120, kde=True, color='green')
    # plt.title("Distribution of Monthly Returns for S&P 500 from 1934-01-31 to 2006-12-29")
    plt.title("Distribution of Monthly Returns for IBM from 2007-01-31 to 2011-12-30")
    plt.xlabel("Monthly Returns")
    plt.ylabel("Frequency")
    plt.show()

q_6()
```

For time period : Jan 1934 to December 2006



For time period : January 2007 to December 2011



7) (Optional): What was the maximum monthly return? Lowest return? For each one of them.

```
def q_7(data_set,name): 2 usages
    max_re = max(data_set)
    min_re = min(data_set)
    print(f"For {name} the maximum monthly return is {max_re} and Lowest return is {min_re}\n")

q_7(Re_sp, name: "S&P")
q_7(Re_ibm, name: "IBM")
```

For S&P the maximum monthly return is 0.25422 and Lowest return is -0.25446

For IBM the maximum monthly return is 0.353799 and Lowest return is -0.261905

8) (Optional): If you miss 10%, of the best months (replacing them by zeros), what would be your wealth at the end of the periods with the \$1,000 investment in Problem 4)?

```

def q_8(data_set, p_name): 2 usages
    #create a copy to store change
    data_set_new = data_set.copy()
    #get the 10% of the top best months
    threshold = data_set.quantile(0.9)
    #set the top 10% return as 0
    data_set_new[data_set_new >= threshold] = 0
    #call q_4 to calculate new result
    q_4(data_set_new, p_name)

q_8(re_sp_ps, p_name: "S&P 500")
q_8(re_ibm_ps, p_name: "IBM")

```

```

invest 1,000$ in S&P 500 from January 2007, at 2011-12-30 will be 601.806849
invest 1,000$ in IBM from January 2007, at 2011-12-30 will be 1192.944527

```

```

Process finished with exit code 0

```

9) (Optional readings) Read the 3 text books below (free at least for now, in the zip file) from time to time to review statistics, regressions, and more advanced statistical models:

Statistics and Analysis of Scientific Data, by Massimiliano Bonamente
 Econometrics, by Badi H. Baltagi
 Statistics and Data Analysis for Financial Engineering, by David Ruppert • David S. Matteson

Sure, working on

10) (Optional readings) Read Cont (2001) for more properties of stock returns, and read further Jiang et al (2018a,b) and Kelly and Jiang (2014) and the references on studies about asymmetry and tail risks.

Sure, working on

11) (Optional, but strongly recommended) Download the return of a favorite stock of yours from WRDS or any other place (then you know how to do other downloads in the future, and show others you are interested in the stock market!).

12)

Query 9068739, for crsp_a_stock

Status:	Success Rerun	
Product:	crsp_a_stock	
Results:	Result Size: 6 KiB Result Count: 60 Rows Output Files: Download .xlsx Output Results are also in your home directory at: ~/web_query_output/. See WRDS Cloud: Access Web Query Output for more details.	
Timing:	Elapsed Time: 4 seconds Submitted: 2024-10-29 23:25 Work Begun: 2024-10-29 23:25 Work Finished: 2024-10-29 23:25	
Input Parameters:	Toggle Input Parameters	

Replace the data file IBM.xlsx by MyFavorite.xlsx and see how its return properties vary as compared with the IBM (adapt both L1_IBM.ipynb and L2_Analysis.ipynb to analyze it), compare its skewness and kurtosis with those of the SP500, and perhaps plot the price path too, to come up with economic stories/explanations on the major up/down trends.

Hints on downloading data:

- 1) WRDS

Login Info (for students taking this class only):

WRDS (Wharton Research Data Services) has an extensive set of data that can be easily downloaded (it is the standard data source for academic research).
<https://wrds-www.wharton.upenn.edu/>

The class account has been setup. You can follow the instructions as below to enroll in the class account.

<https://wrds-www.wharton.upenn.edu/pages/about/wrds-account-types/student-guide-enrolling-in-a-class-account/>

Here is the code you may need during the enrollment process.

Section 1&2: cee3e0e1

Section 3&4: 2323ffbb

Section 5: 548bd08a

Click “Get Data” icon on top; then CRSP; then “Stock/Security Files”, then “Monthly Stock File”. Step 1, choose dates; Step 2, enter the stock ticker, say AAPL; Step 3, select “Holding Period Return” (you may select price too); Step 4, select .xlsx format. Then submit and the download link will pop up shortly.

It will be good to browse WRDS to get a visual view of what is available. Computer automatically direct download is more complex, and sample codes will be provided in the future.

2) Yahoo Finance

Direct computer downloading is easy for a single or a few stock at Yahoo, which is popular, but may not be of high quality as compared with Quandl below. You need to install it before using. See and run the file L2_Data_yfinance.ipynb for details. Downloading a large group of stocks will be more complex and may be discussed in the future.

3) Quandl

Quandl, <https://www.quandl.com/>, is a popular professional data provider, with data on many asset classes and on various regions and various data types (such as sentiment and satellite signals). You need to install and register for a free account to use it (with limits on data features). See and run the file L2_Data_Quandl.ipynb for details.